Fold Trick-Art: Exact Two-Plane Anamorphosis for Printable Sheets and Foldable Screens

Wai-Shing Luk

***Abstract*—A "fold trick-art" sheet is a flat drawing that, once creased and folded through a right angle, reads as a solid three-dimensional object when viewed from one particular point. The craft is a form of anamorphosis, but the anamorphic surface is not a single plane; it is a right-angled corner formed by two planes that meet at the crease. This paper develops the construction as an exact, coordinate-first algorithm. Starting from a viewer V and an object given by planar faces, each object point is projected onto whichever of the two planes its sight ray reaches first; the choice is decided by a single linear form sigma(P) = V_z P_y - V_y P_z, whose sign also determines how every face and edge is split at the crease. The two planes are then unfolded into one sheet, and the result is rendered with a painter's algorithm over exact rational coordinates. We give the projection and unfolding maps, prove that the folded drawing reproduces the object from V, and describe the implementation used to generate printable A4 sheets for a chair, a table and a house. We report a measured exact-integer budget: the largest value formed during the construction is about 2.0e11, so 64-bit integers suffice with about seven orders of magnitude to spare, while 32-bit integers overflow. A line-by-line port to C++ over Fraction reproduces every sheet pixel-for-pixel. Finally we show that the same content, displayed on the two halves of a 90-degree-folded screen, admits rotation: the fold supplies the two planes, and the object can then spin while remaining three-dimensional.**

I. Introduction

A *fold trick-art* sheet is an ordinary piece of paper carrying an apparently distorted drawing. Crease the paper along a marked line, fold it through a right angle, and look from a carefully chosen point: the flat drawing snaps into the appearance of a solid three-dimensional object. The effect is a member of the family of *anamorphoses*, the "curious perspectives" catalogued by Niceron [1] and analysed historically by Baltrusaitis [2]: images that are legible only from one position, the most famous painted example being the skull in Holbein's *The Ambassadors*. Classical anamorphoses, however, use a single reflecting plane (a mirror) or a single viewing plane. The fold-art variants use two planes at right angles, and the fold is precisely what supplies the second plane. The mathematics is elementary projective geometry—central projection onto two planes and an unfolding—but it is easy to get subtly wrong, and hand drawings are, as the practitioners themselves admit, "not quite right" in perspective.

This paper develops the construction as an exact algorithm rather than a drawing. We fix a viewer V, place the object in the corner formed by a floor and a wall that meet at a crease, and project every point of the object onto whichever plane its sight ray meets first. The two planes are then unfolded into a single flat sheet, which is what a printer produces. The method is a direct application of central projection, and every step is kept in exact rational arithmetic so that the output is reproducible and free of the robustness problems that afflict floating-point geometric predicates [3], [4], [5]. We use the same representation throughout as the `projgeom` family of libraries [6], [7], whose points are homogeneous coordinate triples and whose transforms are exact rational matrices [8], [9].

The work described here was carried out interactively and the paper reports the resulting artefacts. Our contributions are the following.

1. A self-contained account of two-plane fold anamorphosis in which the projection, the plane selection, and the splitting of faces at the crease are all consequences of one linear form σ (Section Section III).
2. An exact rendering pipeline—planar-face clipping, crease-split edges, and a square-root-free painter's algorithm—that emits millimetre-accurate A4 sheets (Section Section IV).
3. Three worked objects (a chair, a table, and a house) and the *crease condition* $Y_0 = Z_s V_y / V_z$ that makes a face meet the crease without skew (Section Section V).
4. A measured exact-integer budget for the whole construction, showing that 64-bit integers suffice while 32-bit integers do not, together with a C++ port that reproduces the sheets pixel-for-pixel (Section Section VI).
5. A motion realisation: fold animations, and a foldable-screen display in which the two halves of the screen act as the two planes and the object rotates (Section Section VII).

II. Related Work

Anamorphosis. The oblique anamorphosis of the Renaissance and Baroque is the earliest systematic use of projection to hide an image inside a distorted one; the treatises of Niceron [1] give explicit constructions, and the modern survey of Baltrusaitis [2] documents both oblique and catoptric (mirror) forms. Our construction is oblique throughout—no mirror is involved—but it differs from the classical case in that the projection surface is a corner rather than a plane, and that the corner is produced physically by folding.

Projective geometry. The tools are standard. Central projection is a homography, points and lines are dual, and a projective transformation is fixed by four point correspondences [8], [9], [10]. We use exactly one such map per planar face, computed over the rationals.

Exact and robust computation. The case for exact arithmetic in geometry is long established: naive floating point makes even simple predicates inconsistent [5], while exact rational arithmetic avoids the problem at the cost of an overflow-and-normalisation budget [3]. Adaptive-precision approaches recover speed while remaining exact [4]. Our contribution here is a concrete budget rather than a predicate: we measure how large the rationals actually grow in a complete construction and show that a fixed 64-bit representation is adequate.

Rational trigonometry and the projgeom libraries. The measurement operations used to state the model without transcendental functions follow Wildberger's rational trigonometry [11], and the software follows the design of the `projgeom` project [6], [7], [12].

III. The Two-Plane Model

A. Geometry

Let the room consist of a **floor** $z = 0$ (the region $y > 0$) and a **wall** $y = 0$ (the region $z > 0$), meeting along the **crease**, which we take to be the x-axis. A viewer sits at $V = (V_x, V_y, V_z)$ with $V_y > 0$ (in front of the wall) and $V_z > 0$ (above the floor). The object, given as a collection of planar faces, stands in the corner. Fig. Fig. 1 shows a cross-section in a plane $x = \text{const}$.

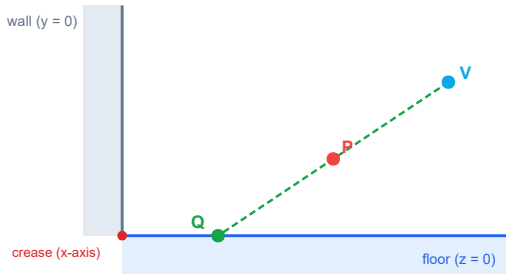


Figure 1: Cross-section of the two-plane model. A sight ray from the viewer V through an object point P meets the floor at Q; the crease is the corner where the wall and floor meet.

B. Projection and the plane that is seen first

Write a ray from V through a point P as $V + t(P - V)$. It meets the floor $z = 0$ and the wall $y = 0$ at the parameters

$$t_{\text{floor}} = \frac{-V_z}{P_z - V_z}, \qquad t_{\text{wall}} = \frac{-V_y}{P_y - V_y}. \tag{1}$$

The visible surface along that ray is the **nearer** plane, i.e. the smaller positive parameter. Comparing the two expressions gives a surprisingly simple criterion: after clearing the (positive) denominators,

$$t_{\text{floor}} < t_{\text{wall}} \iff V_z P_y > V_y P_z \iff \sigma(P) > 0, \tag{2}$$

where

$$\sigma(P) \;=\; V_z\, P_y \;-\; V_y\, P_z. \tag{3}$$

Thus each object point belongs to the **floor flap** when $\sigma(P) > 0$ and to the **wall flap** when $\sigma(P) < 0$; points with $\sigma(P) = 0$ lie on the plane through V and the crease and are seen exactly on the crease. The classifier σ is a *linear* function of P, a fact we exploit below.

C. Unfolding

To print, the two planes are unfolded into one flat sheet by rotating the wall about the crease into the floor plane:

$$(x,\, 0,\, z) \;\longmapsto\; (x,\; z), \qquad (x,\, y,\, 0) \;\longmapsto\; (x,\; -y). \tag{4}$$

In the unfolded sheet the crease is the line $v = 0$; the wall flap occupies $v > 0$ and the floor flap $v < 0$. This is exactly the layout of the physical crafts: the standing part of the object appears above the crease and the flat part below.

D. Why the illusion is exact

The construction is *correct by construction*. Suppose a point P is recorded at Q, the first intersection of the ray $V \to P$ with the corner surface. Fold the sheet back: Q returns to its position in space on the floor or the wall. Because V, Q and P are collinear and Q lies between V and P, the point Q is seen along precisely the same sight line as P. Consequently the drawing, viewed from V, is indistinguishable from the object; it is exact for *any* choice of V on the open side, and does not depend on an approximate perspective. The only requirement is that each object point be assigned to the correct plane, which is what σ decides.

IV. Rendering the Sheet

The objects are polyhedra with planar faces. Three implementation facts make the rendering exact and simple.

A. Faces: whole or split by a line

Because σ is linear, its extreme values over a convex face occur at the vertices. A face therefore lies wholly on one plane if and only if σ has the same sign at all its vertices; if the sign changes, the face is cut by the single line $\sigma = 0$. We clip each straddling face into its wall and floor parts with a half-plane clip (Sutherland–Hodgman on the plane $\sigma = 0$) and project each part onto its own plane. A face that does not straddle is mapped by a single projective transformation, computed from the four point correspondences between the unit square and the projected corners.

B. Edges: bend at the crease

An edge whose endpoints lie on opposite sides of $\sigma = 0$ is split at the crossing and drawn as two segments, so it *bends* at the crease rather than jumping across it.

C. Occlusion: a square-root-free painter's algorithm

Faces are drawn back to front. The depth key is the **squared** distance from V to a face centroid, which is rational and needs no square root; sorting by it and painting far-to-near gives correct visibility for the non-interpenetrating faces used here. This is not a general visibility algorithm: centroid sorting can fail for cyclic overlaps and for interpenetrating or self-occluding faces, which would need a BSP tree or an exact topological visibility method. Outlines are drawn last.

D. Exact arithmetic

Every coordinate is a rational number. The projection of Equation Eq. 1 and the unfolding of Equation Eq. 4 are rational operations, the clip parameter $s_a/(s_a - s_b)$ is rational, and the face homography is computed exactly. The final SVG is written in millimetres with an exact integer rounding, so the sheet is printable at 100%.

V. Objects and the Crease Condition

We generated three objects; Fig. Fig. 2, Fig. 3 and Fig. 4 show the resulting printable sheets. Each sheet is A4 portrait (210 mm × 297 mm) with an explicit dashed crease.

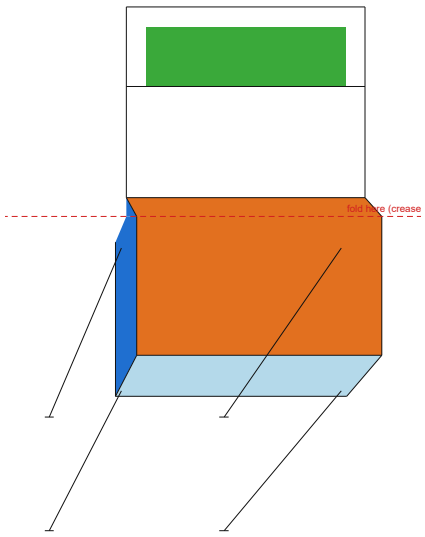


Figure 2: Flat sheet for the chair. Fold along the dashed crease through 90 degrees and view from V to obtain the solid chair.

A. The crease condition

A face placed exactly on the plane through V and the crease—that is, with $\sigma = 0$ at its edge—meets the crease

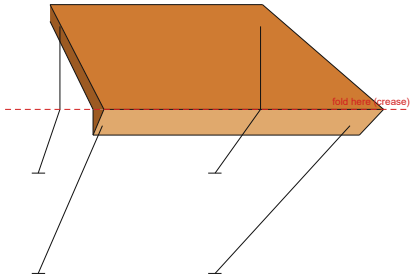


Figure 3: Flat sheet for the table.

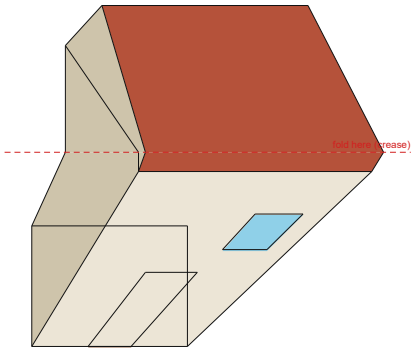


Figure 4: Flat sheet for the house: walls, gable and sloped roof exercise the general planar-face path.

without a gap or a skew. For a backrest whose bottom edge is at height Z_s this gives the **crease condition**

$$Y_0 = \frac{Z_s\, V_y}{V_z}, \tag{5}$$

which is also exactly the condition that the wall-flap scale $t_{\text{wall}} = V_y/(V_y - Y_0)$ equals the floor-flap scale $t_{\text{floor}} = V_z/(V_z - Z_s)$. When it holds, the standing face and the flat face are projected at the same magnification and the drawing is clean; when it fails, one of them is magnified relative to the other and the object appears skewed. This is a genuine design constraint, not a numerical detail: as we found in practice, a "clean backrest" and "the object moved

Table I: Modelling parameters of the three objects (corner coordinates; the viewer is $V = (V_x, V_y, V_z)$).

	chair	table	house
V_x	−120	−130	−150
V_y	280	280	280
V_z	220	230	240
faces	4	3	6
edges	20	16	21
height	0–103	0–74	0–116

further from the wall" are mutually exclusive—moving the object outward must be accompanied by moving the viewer.

Two further consequences follow from σ. First, moving the object *closer* to the wall pushes it to the wall side of $\sigma = 0$, so the crease crosses the flat seat rather than the sharp back corner. Second, because the whole construction is scale-invariant along each sight ray, translating the object and the viewer together leaves the printed sheet unchanged; only their *relative* geometry matters.

B. Parameter summary

Table~I lists the modelling parameters of the three objects. All are small integers or simple fractions, which is what keeps the rationals bounded (Section Section VI).

VI. A C++ Realization and the Integer Budget

The construction was ported, line by line, to the C++ sibling library [7], whose rational type is `fun::Frac`, an alias for `Fraction<std::int64_t>`, and whose four-point homography is `fun::projective_transform`. The port emits the same three sheets and reproduces them pixel-for-pixel: rendering both toolchains' SVGs and comparing them gives an absolute pixel error of zero for all three objects. This is a consistency check between two translations of one algorithm rather than an independent reimplementation: the port follows the same construction step for step, and the Python sources were reorganised to mirror the single C++ file so the two can be maintained together.

A. How large do the rationals get?

Because the `Fraction` type carries no overflow guard, the relevant question is whether a fixed width suffices. We instrumented the exact arithmetic so that the largest numerator or denominator of *every* rational formed during the construction is recorded *before* each reduction—including the internal determinant and adjugate products of the four-point homography, not merely the final coordinates. Table~II reports the results.

The largest value that must be represented is about 2.0×10^{11} (the table), which overflows a 32-bit signed integer (limit 2.1×10^9) by a factor of about 10^2, while it leaves 64-bit integers (limit 9.2×10^{18}) about seven orders of magnitude of headroom. The Python reference, whose `Fraction` reduces identically, stays within the same bound—its largest intermediate homography values are

Table II: Largest exact value formed during the construction (the maximum, over all intermediate rationals, of every numerator and denominator, measured before each reduction), compared with fixed-width limits.

quantity	largest value
chair	9.4×10^7
table	2.0×10^{11}
house	9.6×10^{10}
32-bit limit	2.1×10^9
64-bit limit	9.2×10^{18}

below 10^8. We conclude that **32-bit integers are not sufficient** for this construction, while **64-bit integers are**. This matches the library's own choice of a 64-bit rational and the sibling fraction library's practice of guarding products rather than stored values [12].

VII. Motion: Fold Animations and a Foldable-Screen Display

The sheet is static, but the model is not. Two motions are natural.

Folding. Animating the dihedral angle from 180° (flat) to 90° (folded) while the camera stays at V shows the flat anamorphic drawing rising into the solid object; the reveal occurs at exactly 90°. We render this as a short loop (fold, hold, unfold).

A foldable screen. A foldable phone folded to 90° realises the two planes *in hardware*: one half lies down as the floor plane and the other stands up as the wall plane. Projecting the object onto the two halves from V makes the very same content read as a solid object—and, because the surface is a display rather than paper, the object can **rotate**. Fig. Fig. 5 contrasts the two cases: on a *flat* screen the identical frames appear as a squashed, warping drawing, whereas on the 90°-folded screen they read as a solid, spinning chair. The distinction is exactly Equation Eq. 3: the fold supplies the two planes that the flat screen lacks.

VIII. Discussion and Limitations

A. What the construction does well

Anamorphosis by central projection is exact for *any* viewer position, so the method never needs a special-case perspective; the only design freedom is the viewer and the object. Keeping every step in exact rational arithmetic makes the output reproducible and immune to the inconsistency of floating-point predicates [5], and the same representation carries over verbatim to a compiled language (Section Section VI) and to motion (Section Section VII). The rendering is simple: one linear classifier, one clip line per straddling face, and a painter's sort on a rational key.

B. Limitations

Occlusion is approximate. Visibility is resolved by a centroid painter's algorithm (Section Section IV), which is correct

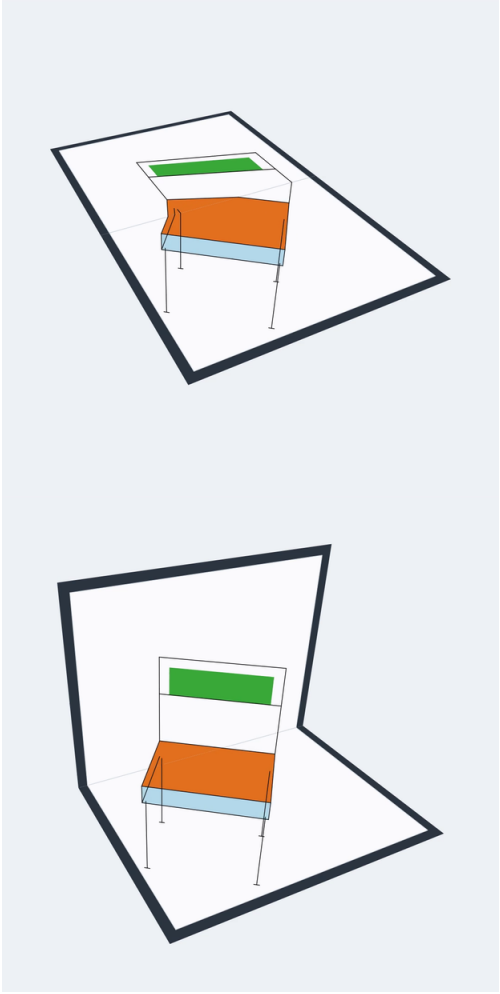


Figure 5: The same rotating-chair content on a flat screen (top) and on a 90°-folded screen (bottom). Only the fold makes the object read as three-dimensional.

only for the non-interpenetrating, individually convex, front-to-back separable faces used here. It fails for cyclic overlaps, for interpenetrating or self-occluding meshes, and for faces of very different aspect ratio; a general scene needs a BSP tree or an exact topological visibility algorithm.

Exactness stops at the rationals. The construction is exact for rational inputs; a face whose coordinates involve roots of non-squares cannot be represented and would require a number field or a symbolic type. *Scale invariance* cuts both ways: it guarantees that moving object and viewer together is invisible, but it also means the printed sheet depends only on the relative geometry, so a "moved" object cannot be shown without also moving the viewer or breaking alignment. *Coordinate growth* is benign here only because the constructions are shallow; long chains of homographies would grow the rationals and eventually require wider or arbitrary-precision integers.

Monocular, single-viewpoint, and unlit. The illusion is strictly monocular and single-viewpoint: binocular disparity exposes the physical corner, and the effect degrades continuously away from V—the further the object from the corner, the sharper the required constraint on the viewer (the trade-off we recorded for the chevron-style constructions). The renderer is also purely geometric—flat shading, with no lighting, shadows or reflectance—whereas in the physical craft the shadows cast across the crease are themselves strong depth cues.

Narrow validation. The framework is exercised by three small objects (four, three and six faces); densely tessellated or scanned models are untested, and the reported budget should be read as a measurement for this family rather than a general bound.

IX. Conclusion

Fold trick-art is anamorphosis onto a corner, and a corner is two planes at a right angle. Once the problem is stated that way, the entire construction reduces to one linear form $\sigma(P) = V_z P_y - V_y P_z$: it decides which plane each point belongs to, where every face is cut, and where every edge bends. The two planes unfold to a printable A4 sheet, the result is exact for any viewer, and the same coordinates carry over to C++ with a measured 64-bit integer budget and to a foldable screen where the fold is provided by the device and the object can spin. We hope the account makes a pleasing craft mathematically transparent and gives a small, fully worked example of coordinate-first, exact-rational computation.

X. Code and Data Availability

The Python generators, printable sheets, and animations are in `projgeom-py` [6]; the exact-rational C++ port is in `projgeom-cpp` [7]; the rational arithmetic is provided by `fractions-cpp` [12].

[1] J.-F. Nicéron, *La perspective curieuse*. Paris: Pierre Billaine, 1638.

[2] J. Baltrusaitis, *Anamorphoses ou thaumaturgus opticus*. Paris: Flammarion, 1976.

[3] C.-K. Yap, *Fundamental problems of algorithmic algebra*. New York: Oxford University Press, 2000.

[4] J. R. Shewchuk, "Adaptive precision floating-point arithmetic and fast robust geometric predicates," *Discrete & Computational Geometry*, vol. 18, no. 3, pp. 305–363, 1997.

[5] L. Kettner, K. Mehlhorn, S. Pion, S. Schirra, and C. Yap, "Classroom examples of robustness problems in geometric computations," *Computational Geometry: Theory and Applications*, vol. 40, no. 1, pp. 61–78, 2008.

[6] W.-S. Luk, "Projgeom-py: Projective geometry in python." https://github.com/luk036/projgeom-py, 2026.

[7] W.-S. Luk, "Projgeom-cpp: Projective geometry in c++." https://github.com/luk036/projgeom-cpp, 2026.

[8] H. S. M. Coxeter, *Projective geometry*, 2nd ed. New York: Springer-Verlag, 1974.

[9] J. Richter-Gebert, *Perspectives on projective geometry: A guided tour through real and complex geometry*. Berlin, Heidelberg: Springer, 2011.

[10] J. Stillwell, *The four pillars of geometry*. New York: Springer, 2005.

[11] N. J. Wildberger, *Divine proportions: Rational trigonometry to universal geometry*. Sydney: Wild Egg Books, 2005.

[12] W.-S. Luk, "Fractions-cpp: Header-only rational arithmetic in c++." https://github.com/luk036/fractions-cpp, 2026.